



COURSE MATERIAL

Jungle Jump Pt. 2

CMSC 197 GDD Activity 6

Prepared by: Rene Andre B. Jocsing

Published: March 17, 2026

Session Objectives

- Implement player movement with velocity-based physics
 - Complete finite state machine with state transitions
 - Integrate animation system with state machine
 - Add gravity and jump mechanics
 - Create player health system with damage response
-

Recap from Pt. 1

Last session, you created the Player scene with:

- CharacterBody2D root node with proper collision layers
- AnimationPlayer with all movement animations
- Camera2D for viewport control
- Basic FSM skeleton with enum states

Today, you'll bring the player to life with movement, state transitions, and a health system.

Player Movement Implementation

Input Processing

The player needs three controls: left, right, and jump. Create a `get_input()` function that processes inputs and determines state transitions.

Add to player.gd (after `change_state` function):

```
1 func get_input() -> void:
2     var right: bool = Input.is_action_pressed("right")
3     var left: bool = Input.is_action_pressed("left")
4     var jump: bool = Input.is_action_just_pressed("jump")
5
6     # Movement occurs in all states
7     velocity.x = 0
8     if right:
9         velocity.x += run_speed
```

```

10     $Sprite2D.flip_h = false
11     if left:
12         velocity.x -= run_speed
13         $Sprite2D.flip_h = true
14
15     # Only allow jumping when on the ground
16     if jump and is_on_floor():
17         change_state(JUMP)
18         velocity.y = jump_speed
19
20     # IDLE transitions to RUN when moving
21     if state == IDLE and velocity.x != 0:
22         change_state(RUN)
23
24     # RUN transitions to IDLE when standing still
25     if state == RUN and velocity.x == 0:
26         change_state(IDLE)
27
28     # Transition to JUMP when in the air
29     if state in [IDLE, RUN] and not is_on_floor():
30         change_state(JUMP)

```

Design Pattern: Velocity Reset

`velocity.x = 0` at the start of `get_input()` ensures the player stops immediately when no keys are pressed. Without this, the player would continue sliding due to retained velocity.

This differs from `RigidBody2D` where friction gradually reduces velocity. For platformers, instant response feels better.

Input Method Comparison

Method	Behavior
<code>is_action_pressed()</code>	Returns true continuously while key is held
<code>is_action_just_pressed()</code>	Returns true only on the frame the key was pressed

Jump uses `is_action_just_pressed()`: Players must press jump each time. Holding the key doesn't create continuous jumps. This prevents accidental double-jumping and provides precise control.

Sprite Flipping

```

1  $Sprite2D.flip_h = false # Face right
2  $Sprite2D.flip_h = true  # Face left

```

Rationale: Instead of rotating the sprite or creating separate left/right animations, we flip the sprite horizontally. This saves memory and animation effort while maintaining visual correctness.

Physics Processing

The Physics Loop

Add the `_physics_process()` function to handle gravity, input, and movement:

```

1 func _physics_process(delta: float) -> void:
2     velocity.y += gravity * delta
3     get_input()
4     move_and_slide()
5
6     # JUMP ends when landing
7     if state == JUMP and is_on_floor():
8         change_state(IDLE)
9
10    # Switch to falling animation
11    if state == JUMP and velocity.y > 0:
12        $AnimationPlayer.play("jump_down")

```

Architecture Note: Physics Frame Independence

Why multiply by delta?

delta is the time elapsed since the last frame (typically ~0.016s for 60 FPS).

Without delta: `velocity.y += 750` would add 750 pixels/frame

- 60 FPS: $750 * 60 = 45,000$ pixels/second
- 30 FPS: $750 * 30 = 22,500$ pixels/second

With delta: `velocity.y += 750 * delta`

- 60 FPS: $750 * 0.016 * 60 = 750$ pixels/second
- 30 FPS: $750 * 0.033 * 30 = 750$ pixels/second

Result: Consistent physics regardless of framerate.

Can you prove this using Riemann Sums? Recall Math 54 and Physics 71 (Euler's Method and the definition of an integral, alongside the definition of a derivative with respect to acceleration and velocity).

The proof is trivial and is left as an exercise to the reader.

Understanding `move_and_slide()`

`move_and_slide()` performs three critical operations:

1. **Moves the body** by the current velocity vector
2. **Detects collisions** with other physics bodies
3. **Slides along surfaces** when collisions occur

Additionally, because Up Direction is set to (0, -1):

- Collisions below the player = floor
- `is_on_floor()` returns true when standing on surfaces
- `is_on_ceiling()` detects overhead collisions

Common Pitfalls

Common mistake: Calling `move_and_slide()` multiple times

Each call moves the character. Multiple calls in one frame = multiple movements = incorrect physics.

Always call `move_and_slide()` exactly once per `_physics_process()` frame.

Jump Animation Transition

```
1 if state == JUMP and velocity.y > 0:
2     $AnimationPlayer.play("jump_down")
```

Logic:

- `velocity.y < 0`: Moving upward (ascending)
- `velocity.y > 0`: Moving downward (falling)

When velocity transitions from negative to positive, switch from `jump_up` to `jump_down` animation. This creates a natural jump arc visual.

State Transition Rules

From State	To State	Condition
IDLE	RUN	<code>velocity.x ≠ 0</code>
RUN	IDLE	<code>velocity.x == 0</code>
IDLE/RUN	JUMP	Jump pressed and <code>is_on_floor()</code>
IDLE/RUN	JUMP	not <code>is_on_floor()</code> (fell off edge)
JUMP	IDLE	<code>is_on_floor()</code>

Design Pattern: State Transition Validation

Notice that jumps only trigger when `is_on_floor()` is true. This prevents mid-air jumping (unless you want double-jump mechanics).

The state in `[IDLE, RUN]` check ensures players can't transition to JUMP from HURT or DEAD states—a critical gameplay constraint.

Testing the Movement

Before adding more features, verify the movement works correctly.

Test Scene Setup

You should still have the Main scene from last session with a `StaticBody2D` platform. If not:

1. Open `Main.tscn` (or create new scene with Node root)
2. Add Player instance (if not present)
3. Add `StaticBody2D` with `RectangleShape2D` collision
4. Stretch collision shape horizontally
5. Position platform below player

Enable Collision Visualization

Menu: Debug → Visible Collision Shapes

This debug setting draws collision shapes during gameplay. Essential for troubleshooting physics issues.

Testing Checklist

Press **F6** (Play Scene) and verify:

1. Player falls and stops on platform (gravity works)
2. `idle` animation plays when standing still
3. Left/Right movement works, sprite flips correctly
4. `run` animation plays while moving
5. Space bar makes player jump
6. `jump_up` animation plays while ascending
7. `jump_down` animation plays while falling
8. Player returns to `idle` after landing
9. Walking off platform edge triggers JUMP state

Common Pitfalls

Player falls through platform?

Check:

- Player collision mask includes `environment` layer
- `StaticBody2D` collision layer is `environment`
- `CollisionShape2D` has a shape assigned
- Visible Collision Shapes is enabled to debug

Player doesn't jump?

Check:

- Input action `jump` exists with Space assigned
 - No typos in `Input.is_action_just_pressed("jump")`
 - Player is actually on the floor (add `print(is_on_floor())` to debug)
-

Player Health System

The player will start with three hearts and lose one each time damaged.

Health Variables and Signals

Add to top of `player.gd` (after `extends` line):

```

1  signal life_changed
2  signal died
3
4  var life: int = 3: set = set_life
5
6  func set_life(value: int) -> void:
7      life = value
8      life_changed.emit(life)
9      if life <= 0:
10         change_state(DEAD)

```

Design Pattern: Setter Functions

Godot 4 property setter syntax:

```
var life: int = 3: set = set_life
```

Whenever `life` is modified (via `life = 2` or `life -= 1`), the `set_life()` function is automatically called.

Benefits:

- Centralized logic for property changes
- Automatic signal emission on value change
- Validation (e.g., clamping health between 0-3)
- Side effects (triggering state changes)

Old Godot 3 syntax: `var life = 3 setget set_life`

Signal Explanation

- **life_changed:** Emitted whenever health changes. UI will connect to this to update heart display.
- **died:** Emitted when life reaches 0. Main game scene will handle game over logic.

Damage Response Function

```
1 func hurt() -> void:
2     if state != HURT:
3         change_state(HURT)
```

Design Decision: Invulnerability During HURT

The `if state != HURT` check prevents damage loops.

Without this check: Enemy collision → `hurt()` → `life -= 1` → still colliding → `hurt()` → `life -= 1`...

Result: Instant death from single touch.

With this check: Once in HURT state, additional damage calls are ignored until state changes back to IDLE.

Enhanced HURT State Behavior

Update the HURT case in `change_state()`:

```
1 HURT:
2     $AnimationPlayer.play("hurt")
3     velocity.y = -200
4     velocity.x = -100 * sign(velocity.x)
5     life -= 1
6     await get_tree().create_timer(0.5).timeout
7     change_state(IDLE)
```

Breakdown:

- `velocity.y = -200`: Bounce upward
- `velocity.x = -100 * sign(velocity.x)`: Knock back opposite to current movement direction
- `life -= 1`: Decrease health (triggers `set_life()`)
- `await...create_timer(0.5)`: Wait 0.5 seconds
- `change_state(IDLE)`: Return to normal state

Architecture Note: `sign()` Function

`sign(x)` returns:

- 1 if $x > 0$
- -1 if $x < 0$
- 0 if $x == 0$

`-100 * sign(velocity.x)` creates knockback:

- Moving right ($\text{velocity.x} > 0$): $-100 * 1 = -100$ (pushed left)
- Moving left ($\text{velocity.x} < 0$): $-100 * -1 = 100$ (pushed right)

This ensures knockback feels natural—you're pushed away from the danger source.

Update DEAD State

```
1 DEAD:
2     died.emit()
3     hide()
```

Emit signal to notify Main scene, then hide the player sprite.

Disable Input During HURT

Add to the beginning of `get_input()`:

```
1 func get_input() -> void:
2     if state == HURT:
3         return
4     # ... rest of function
```

Rationale: Players shouldn't control the character during damage knockback. The early return prevents all input processing when hurt.

Reset Function for Respawn

Add this function:

```
1 func reset(_position: Vector2) -> void:
2     position = _position
3     show()
4     life = 3
5     change_state(IDLE)
```

Usage: Later, the Main scene will call this when starting a level or respawning after death. This centralizes spawn logic.

Common Pitfalls

Why set position directly here?

Earlier, we said never directly set `position` on `CharacterBody2D`. But during spawn/reset, the player isn't moving—they're being placed. This is safe.

Rule of thumb: Use `move_and_slide()` for movement during gameplay. Direct position assignment is acceptable for teleportation/spawning.

Complete Player Script

Your `player.gd` should now look like this:

```
1  class_name Player extends CharacterBody2D
2
3  signal life_changed
4  signal died
5
6  # Movement parameters
7  @export var gravity: int = 750
8  @export var run_speed: int = 150
9  @export var jump_speed: int = -300
10
11 # State machine
12 enum {IDLE, RUN, JUMP, HURT, DEAD}
13 var state: int = IDLE
14
15 # Health
16 var life: int = 3: set = set_life
17
18 func set_life(value: int) -> void:
19     life = value
20     life_changed.emit(life)
21     if life <= 0:
22         change_state(DEAD)
23
24
25 func _ready() -> void:
26     change_state(IDLE)
27
28
29 func change_state(new_state: int) -> void:
30     state = new_state
31     match state:
32         IDLE:
33             $AnimationPlayer.play("idle")
34         RUN:
35             $AnimationPlayer.play("run")
36         JUMP:
37             $AnimationPlayer.play("jump_up")
38         HURT:
39             $AnimationPlayer.play("hurt")
40             velocity.y = -200
41             velocity.x = -100 * sign(velocity.x)
42             life -= 1
43             await get_tree().create_timer(0.5).timeout
44             change_state(IDLE)
45         DEAD:
46             died.emit()
47             hide()
48
49
50 func get_input() -> void:
51     if state == HURT:
52         return
53
54     var right: bool = Input.is_action_pressed("right")
55     var left: bool = Input.is_action_pressed("left")
56     var jump: bool = Input.is_action_just_pressed("jump")
57
58     # Movement occurs in all states
59     velocity.x = 0
60     if right:
```

```

61     velocity.x += run_speed
62     $Sprite2D.flip_h = false
63   if left:
64     velocity.x -= run_speed
65     $Sprite2D.flip_h = true
66
67   # Only allow jumping when on the ground
68   if jump and is_on_floor():
69     change_state(JUMP)
70     velocity.y = jump_speed
71
72   # IDLE transitions to RUN when moving
73   if state == IDLE and velocity.x != 0:
74     change_state(RUN)
75
76   # RUN transitions to IDLE when standing still
77   if state == RUN and velocity.x == 0:
78     change_state(IDLE)
79
80   # Transition to JUMP when in the air
81   if state in [IDLE, RUN] and not is_on_floor():
82     change_state(JUMP)
83
84
85   func _physics_process(delta: float) -> void:
86     velocity.y += gravity * delta
87     get_input()
88     move_and_slide()
89
90   # JUMP ends when landing
91   if state == JUMP and is_on_floor():
92     change_state(IDLE)
93
94   # Switch to falling animation
95   if state == JUMP and velocity.y > 0:
96     $AnimationPlayer.play("jump_down")
97
98
99   func reset(_position: Vector2) -> void:
100     position = _position
101     show()
102     life = 3
103     change_state(IDLE)
104
105
106   func hurt() -> void:
107     if state != HURT:
108       change_state(HURT)

```

Testing the Complete System

Manual Damage Testing

You don't have enemies yet, but you can test the health system manually.

Add to Main.gd (create script if needed):

```

1  extends Node
2

```

```

3 func _ready() -> void:
4     $Player.life_changed.connect(_on_player_life_changed)
5     $Player.died.connect(_on_player_died)
6
7
8 func _on_player_life_changed(new_life: int) -> void:
9     print("Player life: ", new_life)
10
11
12 func _on_player_died() -> void:
13     print("Player died!")
14
15
16 func _input(event: InputEvent) -> void:
17     # Press Enter to test damage
18     if event.is_action_pressed("ui_accept"):
19         $Player.hurt()

```

Test: Press Enter/Return repeatedly. Console should show life decreasing. After three presses, player should disappear and “Player died!” should print.

Verification Checklist

1. Damage creates upward bounce
2. Knockback pushes player away from movement direction
3. hurt animation plays
4. Input disabled during HURT state
5. Life decreases by 1
6. After 0.5s, returns to IDLE state
7. After 3 hits, player dies and disappears

Session Summary

What We Built

- Complete player movement with responsive controls
- Functional FSM with proper state transitions
- Gravity and jump mechanics
- Health system with damage response and knockback
- Signal-based communication for UI integration

Key Architectural Patterns

1. **Velocity-based movement:** Manual gravity + move_and_slide()
2. **Delta-time physics:** Frame-independent movement
3. **FSM state transitions:** Condition-based state changes
4. **Property setters:** Automatic signal emission on value changes
5. **Invulnerability frames:** Preventing damage loops

GDScript Patterns Introduced

- is_on_floor() for platform detection
- sign() for directional logic
- await...create_timer() for timed state changes
- Property setters with : set = func_name
- Early return pattern for conditional logic

Next Session Preview

- Design levels using TileMap node
- Create TileSet with collision and custom data
- Implement camera limits based on level bounds
- Add screen wrapping for continuous worlds
- Build multiple level scenes